

Alumnxs:

- > Genez, Milena – genezmilena@gmail.com
- > Strusiat, Gonzalo – gonzaloostrusiat@gmail.com
- > Lasota, Pablo – lasotapablo01@gmail.com

Trabajo práctico integrador de la materia Programación Orientada a Objetos II que consiste en un sistema de e-commerce sin frameworks, sin persistencia y sin interfaces gráficas. El objetivo es demostrar el correcto uso de principios y patrones de diseño orientado a objetos.

La siguiente documentación fue hecha a medida que se avanzaba y describe decisiones de diseño tomadas a lo largo del desarrollo de la solución. En algunos casos mostramos el código para ejemplificar pero estas clases y/o fragmentos al final crecen y/o cambian.

>> Módulo 2.1: Catálogo de productos

Este módulo cubre la gestión del catálogo compuesto por dos tipos de entidades: productos y paquetes (bundles). Ambos exponen el mismo contrato al cliente del catálogo.

Productos y paquetes unificados

El catálogo tiene productos simples y paquetes (bundles) que agrupan otros items (también pueden ser otros paquetes). Ambos exponen el mismo contrato al cliente. Se necesitaba calcular precios y operar sobre cualquier item sin distinguir si es simple o compuesto.

Solución: Patrón Composite

Sin composite combinar algunos criterios obligaría a una clase por cada combinación posible o usar `if/switch` por cada operador.

Roles:

Component → `CatalogItem`

Leaf → `Product`

Composite → `Bundle`

Se define la interfaz `CatalogItem` como el **component** del patrón. Tanto el **product** (hoja) como el **bundle** (**composite**) la implementan.

```
CatalogItem (interface) // Component
├── Product              // Leaf - hoja
└── Bundle              // Composite (contiene List<CatalogItem>)
```

El precio del `Bundle` se calcula de forma recursiva y uniforme:

```
// Bundle no sabe si sus items son Products u otros Bundles
public double getBasePrice() {
    return items.stream()
        .mapToDouble(i → i.getBasePrice())
        .sum() * (1 - discountRate);
}
```

La recursión sale del polimorfismo. Un **bundle** anidado dentro de otro **bundle** resuelve su precio solo.

Contrato de CatalogItem:

```
public interface CatalogItem {  
    String getName();  
    String getDescription();  
    double getBasePrice();  
}
```

Atributos dinámicos

Se requería que los productos admitan atributos que no son conocidos en tiempo de diseño. En primera instancia, pensamos en el uso de un `Map<String, Object>` y esto nos resolvía el almacenamiento pero no responde a una solución orientada a objetos ni permite extender el comportamiento de una manera limpia.

Primera aproximación a la solución

Definimos una jerarquía de clases con `Attribute<T>` como clase abstracta:

```
Attribute<T> (abstracta)  
├── DoubleAttribute  
├── StringAttribute  
└── BooleanAttribute
```

Cada subclase conoce su tipo y sabe como determinar si tiene un valor válido (`hasValue`), mostrarse como `String` (`showValue`) y compararse con un criterio de búsqueda (`compareTo`)

```
public abstract class Attribute<T> {  
    // ... atributos y getters  
  
    public boolean compareTo(String value) {  
        return getValue().equals(parseValue(value));  
    } // es igual en DoubleAttribute y BooleanAttribute pero StringAttribute hace  
        override de este método  
  
    public abstract boolean hasValue();  
  
    public abstract String showValue();  
  
    protected abstract T parseValue(String value); // es solo accesible desde la clase  
                                                    y las clases que heredan de esta  
}
```

El problema fue que lo construimos esta solución inicialmente pensando que íbamos a necesitar los atributos dinámicos para comparaciones o más adelante, en los filtros (módulo 2.6). Sin un caso de uso de comparación no había razón para mantener la jerarquía de subclases.

Solución: `Attribute<T>` directamente

En la solución definitiva, eliminamos la jerarquía de subclases y usar una sola clase. El tipo `<T>` se resuelve en tiempo de ejecución. Esto nos simplifica el diseño y evita la complejidad de mantener múltiples subclases.

`Product` mantiene una `List<Attribute<?>>` y opera sobre todos los atributos de manera uniforme sin conocer el tipo concreto.

```
// List como atributo en la clase Product  
private List<Attribute<?>> extraAttributes;  
  
// Ejemplo del método que agrega un atributo dinámico al producto  
public void addExtraAttribute(Attribute<?> attribute) {  
    extraAttributes.add(attribute);  
}
```

El uso de `?` en `List<Attribute<?>>` o en `addExtraAttribute(Attribute<?> attribute)` indica que los tributos de la lista y el parámetro recibido pueden ser de cualquier tipo.

Modificamos la clase `Attribute<T>`:

```
public abstract class Attribute<T> {
    private String name;
    private T value;

    public Attribute(String name, T value){
        this.name = name;
        this.value = value;
        validate(); // la validación
    }

    public String getName() {
        return name;
    }

    public T getValue() {
        return value;
    }
}
```

>> Módulo 2.2: Ciclo de vida del pedido

Un pedido cambia de comportamiento según su estado: por ejemplo, las operaciones válidas en BORRADOR no son las mismas que en CONFIRMADO y el enunciado pide que la operación inválida lance una excepción.

State

Roles:

Context → **Order**: delega su comportamiento al estado actual

State (abstract) → **State**

ConcreteState → Draft, Confirmed, InPreparation, Sent, Delivered, Canceled

En la clase `State`:

Por defecto, TODA operación es inválida. Cada estado sobrescribe solo las operaciones válidas para ellos.

```
public abstract class State {
    public abstract String getName(); // cada state sobrescribe el nombre

    public void addItem(Order order, CatalogItem item, int quantity) {
        throw new IllegalStateException("Error: No se pueden agregar items en el estado: "
            + getName());
    }

    public void confirm(Order order) {
        throw new IllegalStateException("Error: No se puede confirmar el pedido en el
            estado: " + getName());
    }

    public void cancel(Order order) { /* ... misma lógica... */ }
    public void start(Order order) { /* ... */ }
    public void send(Order order) { /* ... */ }
    public void deliver(Order order){ /* ... */ }
}
```

Si fuese una interfaz, cada uno de los seis estados tendría que implementar TODOS los métodos repitiendo el mismo throw en cada operación que no le corresponde. Con esta solución, los estados terminales, por ejemplo no sobrescriben nada

```
public class Delivered extends State {
    @Override public String getName() { return "ENTREGADO"; }
    // ningún otro método: toda operación hereda el throw por defecto
}
```

En Order:

```
public class Order {
    private State state;

    public void confirm() { // Order nunca pregunta si es válido
        state.confirm(this);
    }

    public void cancel() {
        state.cancel(this);
    }

    public void start() {
        state.start(this);
    }

    public void send() {
        state.send(this);
    }

    public void deliver() {
        state.deliver(this);
    }

    public void setState(State newState) {
        this.state = newState;
    }
}
```

Decisión: atomicidad en la validación de stock

Con dos items en el pedido, si el primero tenía stock suficiente y el segundo no, el foreach ya había decrementado el stock antes de ver el segundo.

El pedido quedaba sin confirmar (la excepción interrumpe antes de setState) pero el stock del primer ítem ya estaba alterado.

Corrección:

```
@Override
public void confirm(Order order) {
    if (!order.isEverythingInStock()) {
        throw new IllegalArgumentException("Error: no hay stock suficiente para confirmar el pedido");
    }
    order.getItems().forEach(i → i.getItem().decreaseStock(i.getQuantity()));
    order.setState(new Confirmed());
}
```

Ahora la operación es atómica: o se confirma todo el pedido o no se decrementa absolutamente nada.

Demanda combinada: el mismo producto, suelto y dentro de un bundle

Un caso borde real que vimos haciendo los tests: el pedido tiene un producto suelto y un bundle que contiene ese mismo producto. Preguntar el stock de cada ítem por separado (item.hasStock(cantidad)) no detecta que la demanda total combinada puede superar el stock disponible.

Solución: acumular demanda por SKU antes de validar:

```
// en CatalogItem (interface): cada implementación sabe acumular su propia demanda
void accumulateProductDemand(int multiplier, Map<String, Integer> demandBySku);
boolean hasEnoughStockFor(Map<String, Integer> demandBySku);
```

```
// en Product
@Override
public void accumulateProductDemand(int quantity, Map<String, Integer> demandBySku) {
    // si el map que me pasan por parametro me tiene a mi como clave entonces ...
    if (demandBySku.containsKey(getSku())) {
        // me traigo el valor osea la cantidad
        int actualValue = demandBySku.get(getSku());
        // y aca sumo los valores
        demandBySku.put(getSku(), actualValue + quantity);
    } else { // sino simplemente no suma y queda el valor existente
        demandBySku.put(getSku(), quantity);
    }
}

@Override
public boolean hasEnoughStockFor(Map<String, Integer> demandBySku) {
    return hasStock(demandBySku.get(this.sku));
}
```

```
// en Bundle - delega recursivamente, el mismo Composite que en el módulo anterior
@Override
public void accumulateProductDemand(int quantity, Map<String, Integer> demandBySku) {
    items.forEach(i → i.accumulateProductDemand(quantity, demandBySku));
}

@Override
public boolean hasEnoughStockFor(Map<String, Integer> demandBySku) {
    return items.stream().allMatch(i → i.hasEnoughStockFor(demandBySku));
}
```

```
// en Order
public boolean isEverythingInStock() {
    Map<String, Integer> demand = totalDemandPerSku();
    return items.stream().allMatch(i → i.getItem().hasEnoughStockFor(demand));
}
```

Si un kit contiene 1 jeringa y el pedido también tiene 30 jeringas sueltas, la demanda acumulada por SKU detecta el total real antes de aprobar la confirmación. Esto es algo que había que validar ítem por ítem.

Reglas de reembolso por transición de cancelación

- De BORRADOR a CANCELADO: No repone stock, nunca se decrementó
- De CONFIRMADO a CANCELADO: Se repone stock
- De EN_PREPARACION a CANCELADO: Se repone stock, salen 2 notas de crédito: una por productos y otra por envío
- De ENVIADO a CANCELADO: 1 nota de crédito solo por productos

>> Módulo 2.3: Envío

El sistema ofrece distintos métodos de envío (express, estándar y retiro en sucursal), cada uno con su propia forma de calcular el costo y tiempo de espera. Order necesita el costo final sin conocer los detalles de como se calcula cada variante.

Patrón Strategy

Roles:

Context → Order

Strategy → ShippingType

ConcreteStrategy → ExpressShipping, StandardShipping, LocalPickUp

Solución:

ShippingType es la interfaz Strategy. Order mantiene una referencia a la estrategia elegida y delega:

```
public interface ShippingType {
    double cost(Order order); // costo de las operaciones
    String waitingDays(Order order);
}

// Order no sabe ni le importa qué tipo de envío es
public double getShippingCost() {
    return shippingType.cost(this);
}
```

Inversión de Dependencias hacia los sistemas externos

El problema que teníamos era que la primera versión que hicimos de ExpressShipping y StandardShipping llamaba directo a una clase concreta:

```
// Inicialmente estaba acoplado a una implementación concreta
public double cost(Order order) {
    return EnvioExpressMock.calcularCosto(order.totalCost());
}
```

Entonces, avanzamos con el trabajo hasta el módulo 2.4 que es el módulo de pagos. Al realizar esa solución, volvimos a revisar esta porque ahí habíamos aplicado una regla (creemos) correctamente, por ejemplo, ahí hicimos esto: CreditCardPayment depende de la interfaz CreditCardAPI, no de un banco concreto.

La corrección:

Se definieron interfaces para los dos métodos que dependen de un sistema externo:

```
public class ExpressShipping implements ShippingType {
    private final ExpressShippingAPI apiConnection;

    public ExpressShipping(ExpressShippingAPI apiConnection) {
        this.apiConnection = apiConnection;
    }

    @Override
    public double cost(Order order) {
        return apiConnection.calculateCost(order.totalCost());
    }

    @Override
    public String waitingDays(Order order) {
        return "1 día hábil";
    }
}
```

Y las estrategias reciben la API por constructor

```
public class StandardShipping implements ShippingType {
    private final StandardShippingAPI apiConnection;

    public StandardShipping(StandardShippingAPI apiConnection) {
        this.apiConnection = apiConnection;
    }

    @Override
    public double cost(Order order) {
        return apiConnection.estimateCost(order.totalWeight(), order.getAddress());
    }

    @Override
    public String waitingDays(Order order) {
        return "5 a 7 días hábiles";
    }
}
```

Integración con el resto del dominio

`getShippingCost()` se usa en dos lugares ya existentes, sin que ninguno necesite cambios por esta refactorización:

- `order.getTotalToPay()`: suma el envío al costo de productos para calcular lo que efectivamente se le cobra al medio de pago (módulo 2.4)
- `inPreparation.cancel()`: genera una nota de crédito separada por el costo de envío, distinguible de la nota de crédito por productos

`StandardShipping` además depende de `order.totalWeight()`, que ya delega recursivamente en el `Composite` del catálogo (`bundle.getWeight()` suma el peso de cada ítem, incluso si hay otros bundles anidados), entonces, ninguna parte del envío necesita preocuparse por esa recursión.

>> Módulo 2.4: Métodos de pago

El enunciado describe un proceso con pasos y orden fijos: **validar, reservar, ejecutar y notificar**. Acá, cada medio de pago (tarjeta, transferencia, billetera virtual) implementa esos pasos de forma distinta.

Patrón: Template Method

Roles:

AbstractClass → `PaymentMethod`

ConcreteClass → `CreditCardPayment`, `BankTransferPayment`, `VirtualWalletPayment`

```
public abstract class PaymentMethod {
    public final void process(Order order) { // final
        validateData(order);
        setAsideFunds(order);
        executeTransaction(order);
        notifyResult(order);
    }

    public abstract void validateData(Order order);
    public abstract void setAsideFunds(Order order);
    public abstract void executeTransaction(Order order);

    public void notifyResult(Order order) {
        // hook con comportamiento default
    }
}
```


Sobre **notifyResult**: el hook por defecto registra el recibo (PaymentReceipt) con el código de operación y el monto total. Esto ya satisface "generarlo y registrarlo" para tarjeta y transferencia sin que esas clases necesiten sobreescribir nada

Interfaces como contrato – no implementadas

CreditCardAPI, **BankTransferAPI** y **VirtualWalletAPI** representan sistemas externos (el banco emisor, el sistema de transferencias, el proveedor de la billetera, no sabemos). El dominio de e-commerce no controla esos sistemas, solo necesita un **contrato** para usar.

```
public interface CreditCardAPI {  
    void validateCard(String cardNumber, String cvv, String expirationDate);  
    String preAuthorize(double amount);  
    void charge(double amount, String operationNumber);  
}
```

Principio de inversión de dependencias

CreditCardPayment depende de la abstracción CreditCardAPI, no de un banco concreto. El día que haya que integrar un banco real, se escribe una clase que implemente la interfaz y el dominio no cambia.

Caso particular:

El número de operación lo devuelve la API externa en el momento de pre-autorizar o transferir. Por eso executeTransaction puede usar el código generado en el paso anterior (setAsideFunds), aprovechando que Template Method garantiza el orden en process() como final (nadie puede alterarlo)

```
// esto en CreditCardPayment  
@Override  
public void setAsideFunds(Order order) {  
    setOperationNumber(apiConnection.preAuthorize(order.getTotalToPay()));  
}  
  
@Override  
public void executeTransaction(Order order) {  
    apiConnection.charge(order.getTotalToPay(), getOperationNumber());  
}
```

Integración con el ciclo de vida del pedido (State)

En el estado **BORRADOR** (clase Draft), el método confirm() es el unico lugar desde donde se dispara el pago:

```
@Override  
public void confirm(Order order) {  
    if (!order.isEverythingInStock())  
        throw new IllegalArgumentException("Error: no hay stock suficiente para confirmar el pedido");  
  
    order.getPaymentMethod().process(order); // si falla, lanza error acá  
  
    order.getItems().forEach(i → i.getItem().decreaseStock(i.getQuantity()));  
    order.setPaymentReceipt(order.getPaymentMethod().getReceipt());  
    order.setState(new Confirmed());  
}
```

El orden de estas tres acciones importa:

1. **Verificar stock primero**: evita cobrarle a alguien por algo que no hay
2. **Procesar el pago**: si la tarjeta es rechazada process() lanza una excepción y ni el stock ni el estado del pedido se tocan

3. **Decrementar stock y cambiar el estado:** SOLO si el pago fue exitoso, el pedido nunca queda en un estado a medio confirmar

>> Módulo 2.5: Notificaciones del pedido

En este módulo se usa **Observer** (Order avisa cambios de estado sin conocer sus suscriptores).

Roles:

Subject → Order

Observer → OrderObserver

ConcreteSubject → Order (mismo)

Que alternativas evaluamos y como llegamos a la solución final:

Primera versión: instanceof en cada observador

```
public class EmailNotifier implements OrderObserver {
    @Override
    public void onStateChanged(Order order, State prev, State next) {
        if (!aplica(next)) return;
        mailSender.enviarMail(order.getEmail(), "Tu pedido cambió de estado",
            "Ahora está: " + next.getName(), null);
    }

    private boolean aplica(State next) {
        return next instanceof Confirmed
            || next instanceof Sent
            || next instanceof Delivered;
    }
}
```

El tema acá es que **el estado ya está modelado**. Invertimos tiempo en modelar State como una jerarquía de clases reales precisamente para tener objetos que se comportan según su propio tipo. Que un objeto tenga que preguntar con **instanceOf** para decidir algo es raro: el objeto debería poder responder por si mismo.

Opción 1: el estado decide a quien notificar

```
// el estado decidiría CUÁNDO notificar y a QUIÉN
public class Confirmed extends State {
    @Override
    public void start(Order order) {
        order.setState(new InPreparation());
        order.notifyEmailSubsystem(order); // State pasa a SABER que existe "email"
    }
}
```

Descartamos esta. Para que State decida "avisarle a un suscriptor en específico", State necesita conocer los subsistemas concretos uno por uno.

Si aparece un cuarto observador, esta opción obligaría a volver a tocar Confirmed, Sent y Delivered para agregarles la llamada correspondiente. Acoplamiento directo entre el estado y cada subsistema concreto.

Opción 2: un flag genérico esNotificable()

```
public abstract class State {
    // ...
    public boolean esNotificable() { return false; }
}
```

Cada observador necesita una pregunta distinta y un solo booleano no podría responder las tres preguntas.

Si `Confirmed.esNotificable()` devuelve `true`, `Order` notificaría a los tres observadores por igual, pero `InvoiceGenerator` y `LoyaltyProgram` no deberían reaccionar ante `Confirmed`. La pregunta "¿es relevante para mí?" depende de **quién pregunta** y un único `bool` no es tan específico.

La solución elegida: varios predicados con significado de dominio

```
public abstract class State {
    // ... el código existente acá
    public boolean isCancelled() {
        return false;
    }
    public boolean isSuccessfulProgress() {
        return false;
    }
    public boolean isFinal(){
        return false;
    }
}
```

```
public class Confirmed extends State {
    // ... el código existente acá
    @Override
    public boolean isSuccessfulProgress() {
        return true;
    }
}
```

```
public class Sent extends State {
    // ... el código existente acá
    @Override
    public boolean isSuccessfulProgress() {
        return true;
    }
}
```

```
public class Delivered extends State {
    // ... el código existente acá
    @Override
    public boolean isSuccessfulProgress() {
        return true;
    }
    @Override
    public boolean isFinal() {
        return true;
    }
}
```

```
public class Canceled extends State {
    // ... el código existente acá
    @Override
    public boolean isCancelled() {
        return true;
    }
}
// la clase Draft y la clase InPreparation no sobrescriben nada-- heredan los tres false
```

Y los observadores dejan de preguntar **tipo**, preguntan **significado**:

```
public class EmailNotifier implements OrderObserver {
    @Override
    public void onStateChanged(Order order, State prev, State next) {
        if (next.isSuccessfulProgress()) {
            mailSender.enviarMail(order.getEmail(), "Tu pedido cambió de estado",
                "Ahora está: " + next.getName(), null);
        }
    }
}
```

y así con todas...

>> Módulo 2.6: Búsqueda

Esta es la misma estructura que se usó para `CatalogItem/Product/Bundle` en el módulo 2.1 pero aplicada a un problema distinto: en vez de combinar productos para sumar precios, se combinan condiciones booleanas para evaluar si un item hace match en una búsqueda.

Patrón: Composite

Sin composite combinar algunos criterios obligaría a una clase por cada combinación posible o usar `if/switch` por cada operador.

Roles:

Component → `SearchCriteria` (interfaz)

Leaf → `NameContainsCriteria`, `MaxPriceCriteria`, `CategoryCriteria`, `AvailabilityCriteria`

Composite → `AndCriteria`, `OrCriteria`, `NotCriteria`

Catalog → `Catalog`

```
public interface SearchCriteria {
    boolean isSatisfiedBy(CatalogItem item);
}
```

Por qué NOT tiene una forma distinta a AND y OR

```
public class AndCriteria implements SearchCriteria {
    private final List<SearchCriteria> criteria;
}
```

```
public class NotCriteria implements SearchCriteria {
    private final SearchCriteria criteria;
}
```

NOT contiene un solo criterio porque negar es una operación de un solo argumento.

La recursión:

```
public class AndCriteria implements SearchCriteria {
    @Override
    public boolean isSatisfiedBy(CatalogItem item) {
        return criteria.stream().allMatch(c -> c.isSatisfiedBy(item));
    }
}
```

Si uno de los criterios de la lista es, a su vez, otro `OrCriteria` o `AndCriteria`, la llamada a `isSatisfiedBy` se resuelve sola sin que este código sepa ni le importe que tanto esté anidado.

Cambios mínimos al dominio existente

```
public interface CatalogItem {  
    // ... todo lo existente ...  
    String getCategory(); // nuevo  
}
```

Product ya tenía el **getCategory()** con esa firma exacta, solo se le agregó el **override**.

```
public class CategoryCriteria implements SearchCriteria {  
    @Override  
    public boolean isSatisfiedBy(CatalogItem item) {  
        return category.equalsIgnoreCase(item.getCategory());  
    }  
}
```

Catalog → el client del patrón

```
public class Catalog {  
    private final List<CatalogItem> items = new ArrayList<>();  
  
    public void addItem(CatalogItem item) {  
        items.add(item);  
    }  
  
    public List<CatalogItem> search(SearchCriteria criteria) {  
        return items.stream()  
            .filter(criteria::isSatisfiedBy)  
            .collect(Collectors.toList());  
    }  
}
```

Hasta este momento, el catálogo como concepto de negocio nunca tuvo un objeto propio. El enunciado lo nombra explícitamente: *"la búsqueda opera sobre la colección de items del catálogo"*

Catalog encapsula esa colección igual que Order encapsula su lista de OrderItem o Bundle encapsula su lista de ítems internos → nadie manipula la lista desde afuera

>> Módulo 2.8: Reportes

El propio enunciado indica el patrón: **Visitor**.

Patrón aplicado: Visitor

Roles:

Element → Report (interfaz)

ConcreteElement → ProductSalesReport (y futuros tipos de reporte)

Visitor → ReportVisitor (interfaz)

ConcreteVisitor → PlainTextExporter, CsvExporter, HtmlExporter

```
public interface Report {  
    String accept(ReportVisitor visitor);  
}  
  
public interface ReportVisitor {  
    String visit(ProductSalesReport report);  
}
```

En el enunciado dice: *"cada reporte puede exportarse en más de un formato"* y nos dan 3 (texto, CSV, HTML) pero en algún momento, esto podría crecer y tener otros formatos.

La idea es que se pueda sumar los que queramos en un futuro sin tocar ningún reporte existente. Hay una operación real (generate) que varía por tipo y ese conjunto de operaciones es el que se espera que crezca.

Clases nuevas (paquete `ecommerce.catalog.reports`)

```
public class SalesEntry {
    private final CatalogItem item;
    private final int unitsSold;
    private final double averagePricePaid;
    // constructor + getters
}
```

```
public class ProductSalesReport implements Report {
    private final List<SalesEntry> entries;

    @Override
    public String accept(ReportVisitor visitor) {
        return visitor.visit(this);
    }
}
```

```
public class ProductSalesReportGenerator {
    public ProductSalesReport generate(List<Order> orders) {
        // ...
        return new ProductSalesReport(entries);
    }
}
```

```
public class PlainTextExporter implements ReportVisitor { /* ... */ }
public class CsvExporter implements ReportVisitor { /* ... */ }
public class HtmlExporter implements ReportVisitor { /* ... */ }
```

Cada generador implementa `visit(ProductSalesReport)` con su propio formato de salida. Se puede agregar un cuarto formato es una clase nueva, sin tocar `ProductSalesReport`, `SalesEntry` ni el generador.